How-To Library

Informatica

Sizing Guidelines and Performance Tuning for Big Data Streaming 10.2.2

© Copyright Informatica LLC 2019, 2022. Informatica, the Informatica logo, and Big Data Management are trademarks or registered trademarks of Informatica LLC in the United States and many jurisdictions throughout the world. A current list of Informatica trademarks is available on the web at https://www.informatica.com/trademarks.html. Other company and product names may be trade names or trademarks of their respective owners.

Abstract

You can tune Big Data Streaming for better performance. This article provides recommendations that you can use to tune hardware, Spark configuration, mapping configuration, and the Kafka cluster.

Supported Versions

- Informatica Big Data Streaming 10.2.2

Table of Contents

Overview. . . . 2
1. Determine the Deployment Type. . . . 3
Tune the Kafka Cluster. . . . 3
Deployment Criteria. . . . 3
Deployment Type Comparison. . . . 3
2. Determine Hardware Requirements. . . . 4
3. Tune the Spark Engine. . . . 4
4. Tune the Mapping. . . . 6
Sizing Recommendations for Overhead Properties. . . . 6
Stateful Computing. . . . 7
Executor and Driver Logging 7
5. Optimize Transformations. . . . 7
Lookup Transformation. . . . 7
Recommendations for Tuning the Kafka Cluster. . . . 8
Troubleshooting Kafka Errors. . . . 8
Troubleshooting Spark Driver Errors. . . . 9
Documentation Reference. . . . 9

Overview

Use Informatica Big Data Streaming mappings to collect streaming data, build the business logic for the data, and push the logic to a Spark engine for processing. The Spark engine uses Spark Streaming to process data. The Spark engine reads the data, divides the data into micro batches, and publishes it.

Streaming mappings run continuously. When you create and run a streaming mapping, a Spark application is created on the Hadoop cluster which runs forever unless killed or cancelled through the Data Integration Service.

To optimize the performance of Big Data Streaming and your system, perform the following tasks:

1. Determine the deployment type
2. Determine hardware requirements
3. Tune the Spark engine
4. Tune the mapping
5. Optimize transformations

1. Determine the Deployment Type

Sizing and tuning recommendations vary based on the deployment type.

Based on certain deployment factors in the domain and Hadoop environments, Informatica categorizes Big Data Streaming into the following types:

- Sandbox deployment
- Small deployment
- Medium deployment
- Large deployment

Tune the Kafka Cluster

The following table lists the Kafka cluster properties that you can tune based on the deployment type :

Property	Sandbox	Small	Medium	Large
Number of Kafka brokers for each node	1	2	4	8
Number of Kafka nodes	1	2	4	8

Note: Kafka brokers can be consumers or producers.

Deployment Criteria

The following criteria determine the Big Data Streaming deployment type:

Number of messages processed

The total number of messages per second processed from a well-tuned Kafka cluster and written back to a well-tuned Kafka cluster.

Total cores

The total number of cores for each container.

Total memory

The maximum RAM available for each container.

Deployment Type Comparison

The following table compares Big Data Streaming deployment types based on the standard values for each deployment factor:

Deployment	Messages Per Second	CPU	Memory
Sandbox	100,000	- Domain. 4 - Hadoop. 24	- Domain. 16 GB - Hadoop. 40 GB
Small	500,000	- Domain. 4 - Hadoop. 64	- Domain. 16 GB - Hadoop. 120 GB

Deployment	Messages Per Second	CPU	Memory
Medium	1 million	- Domain. 4 - Hadoop. 116	- Domain. 16 GB - Hadoop. 224 GB
Large	10 million	- Domain. 4 - Hadoop. 1056	- Domain. 16 GB - Hadoop. 2104 GB

2. Determine Hardware Requirements

To optimize the performance, acquire the right type of hardware for your Big Data Streaming environment.

Consider the following points while determining your hardware:

- The criteria for determining a deployment is the number of messages processed and not the size of data in gigabytes.
- Choose the hardware such that batches do not get queued. Batches are queued when the batch processing time is greater than the batch interval time.
- Use the following approach to determine hardware capacity:
  - Implement a proof of concept (POC) to determine the throughput for each core.
  - Determine the peak load in terms of the anticipated number of the messages processed per second.
  - Divide the peak load by the throughput for each core to get the total number of cores required and the size of the disk, memory, and network appropriately.

3. Tune the Spark Engine

To optimize Big Data Streaming performance, you must tune Spark parameters. To tune the parameters, configure the Hadoop connection advanced properties for the Spark engine in the Developer tool or the Administrator tool. To tune the parameters for specific mappings, configure the execution parameters of the streaming mapping run-time properties in the Developer tool.

You can configure the following parameters based on the input data rate, mapping complexity, and concurrency of mappings:

spark.executor.cores

The number of cores that each executor process uses to run tasklets.

Recommended value: Specify 3 to 4 cores for each executor. Specifying a higher number of cores might lead to performance degradation.

spark.executor.instances

The number of instances that each executor process uses to run tasklets.

Configure the number of executor instances based on the following deployment types:

- Sandbox deployment. 4
- Small deployment. 14
- Medium deployment. 27
- Large deployment. 262

Note: For mapping with multiple targets, you can configure the `spark.executor.instances` and `spark.executor.cores` parameters such that the sum of its values is equal to the total number of cores required for processing targets in a mapping. For example, consider a mapping with two targets, Target1 and Target2. If Target1 requires 10 cores and Target2 requires 15 cores, then the total number of cores required by the mapping is 25. Configure the `spark.executor.instances` and `spark.executor.cores` parameters such that the sum of its values equals to 25.

spark.executor.memory

The amount of memory that each executor process uses to run tasklets.

Recommended value: Specify a value of 8 GB.

spark.driver.memory

The driver process memory that the Spark engine uses to run mapping jobs.

Recommended value: Specify a value of 8 GB.

spark.driver.cores

The number of cores to use for each driver process.

Recommended value: Specify 8 cores.

spark.sql.shuffle.partitions

The number of partitions that the Spark engine uses to shuffle data to process joins or aggregations in a mapping job.

Recommended value: Specify a value that equals the total number of executor cores if total executor cores allocated is less than 200. Maximum value is 200.

Configure the partitions based on the following deployment types:

- Sandbox deployment. 16
- Small deployment. 56
- Medium deployment. 108
- Large deployment. 200

spark.kryo.registrationRequired

Indicates whether registration with Kryo is required.

Recommended value: True

spark.kryo.classesToRegister

The comma-separated list of custom class names to register with Kryo if you use Kyro serialization.

Specify the following value for all deployment types:

```
org.apache.spark.sql.catalyst.expressions.GenericRow,[Ljava.lang.Object;,
org.apache.spark.sql.catalyst.expressions.GenericRowWithSchema,
org.apache.spark.sql.types.StructType,[Lorg.apache.spark.sql.types.StructField;,
org.apache.spark.sql.types.StructField,
org.apache.spark.sql.types.StringType$,
org.apache.spark.sql.types.Metadata,
scala.collection.immutable.Map$EmptyMap$
[Lorg.apache.spark.sql.catalyst.InternalRow;,
scala.reflect.ClassTag$$anon$1,java.lang.Class
```

infaspark.lookup.persist.enabled

Enables lookup data caching.

To increase the performance of cached Hive lookup, set this parameter to true.

infaspark.lookup.repartition.partitions

Partitions lookup data for better performance.

To increase the performance of cached Hive lookup, set this parameter to any value greater than 1.

infaspark.lookup.hbase.batchsize

The number of records fetched at a time in the HBase lookup database.

Default value: 1500

ExecutionContextOptions.Spark.StreamingDropEmptyBatches

To prevent Spark from creating jobs and tasks when there are no messages to be processed in a batch, set this parameter to true. You can configure this property in the **Custom Properties** tab of the Data Integration Service.

4. Tune the Mapping

To tune the mapping, use the sizing recommendations based on mapping complexity.

Mapping complexity is defined by the number of sources, targets, and transformations in the mapping. Transformations can be CPU bound (Expression transformation), memory bound (Lookup transformation), or disk bound.

Mappings can be grouped into following categories of complexity:

Complexity	Sources	Transformations	Targets
Simple	3	6 (3 memory bound and 3 CPU bound)	1
Standard	7	10 (8 memory bound and 2 CPU bound)	1
Complex	6	13 (8 memory bound and 3 CPU bound)	1

To increase mapping performance, use Kafka headers. Some of the uses of Kafka headers include filtering, routing and schema detection. When you use Kafka headers, there is no need to deserialize messages.

Sizing Recommendations for Overhead Properties

The following table lists the overhead requirements based on mapping complexity:

Mapping Complexity	VCPUs Overhead	Memory Overhead
Simple	33%	33%
Standard	120%	120%
Complex	150%	150%

Stateful Computing

Stateful computing involves storing and retrieving state while evaluating expressions in an Expression transformation.

Configure partitioning by specifying one or more columns under partition keys on the **Windowing** tab.

Configure the partition keys to define partition boundaries, rather than performing the calculation across all inputs.

If you do not define partition keys, all the data is included in the same partition. The variable values stored will be global and every row can read and update the same set of variables.

To improve mapping performance, configure partition keys.

Executor and Driver Logging

You can configure the Data Integration Service to log executor and driver logs.

You can configure the following tracing levels for the logs:

Tracing Level	Messages
None	The log displays FATAL messages. FATAL messages include non-recoverable system failures that cause the service to shut down or become unavailable.
Terse	The log displays FATAL and ERROR code messages. ERROR messages include connection failures, failures to save or retrieve metadata, service errors.
Normal	The log displays FATAL, ERROR, and WARNING messages. WARNING errors include recoverable system failures or warnings.
Verbose initialization	The log displays FATAL, ERROR, WARNING, and INFO messages. INFO messages include system and service change messages.
Verbose data	The log displays FATAL, ERROR, WARNING, INFO, and DEBUG messages. DEBUG messages are user request logs.

5. Optimize Transformations

You can optimize transformations to enable the Spark engine to process transformations in a mapping efficiently.

Lookup Transformation

Consider the following restrictions when you optimize the Lookup transformation to run on the Spark engine:

Data skew

Data skew refers to uneven distribution of data. Spark engine optimization might lead to data skew among executors due to the location of the data. To avoid such an issue, you can set the `spark.shuffle.reduceLocality.enabled` property to false.

Note: When the `spark.shuffle.reduceLocality.enabled` property is set to false, the shuffle behaviour is impacted.

Inefficient lookup partitioning

Mapping performance might degrade due to inefficient lookup partitioning and caching.

To configure cache partitioning for a lookup transformation, perform the following steps:

- Set the value of `infaspark.lookup.repartition.partitions` property equal to the number of source topic partitions. For example, if a Kafka topic has 18 partitions, set the value of `infaspark.lookup.repartition.partitions` property to 18.
- Set the value of `infaspark.lookup.persist.enabled` property to true.

Data duplication

Avoid data duplication in lookup source. If lookup data is unique, configure to return all rows on multiple matches.

Recommendations for Tuning the Kafka Cluster

Consider the following recommendations to tune the Kafka cluster:

- Configure the Kafka cluster so that Big Data Streaming can produce and consume messages at the needed message ingestion rate.
- To increase the rate of message consumption in Big Data Streaming, increase the number of Kafka brokers in the Kafka cluster and in the Kafka connection.
- Increase the number of partitions on the Kafka topic. Ideally, the number of partitions can be equal to the number of CPU cores allocated to the executors. For example, if you set `spark.executor.instances` to 6 and `spark.executor.cores` to 3, there are 18 cores allocated. Then set the number of Kafka partitions to 18, so that there are 18 parallel tasks to read from the Kafka Source.
  For example, you can use the following command to specify the number of partitions:

  ```
  ./ kafka-topics.sh --create --zookeeper
  zookeeper_host_name1:zookeeper_port_number ,zookeeper_host_name2:zookeeper_port_number,zoo
  keeper_host_name3:zookeeper_port_number --replication-factor 1 --partitions 18 --topic
  NewOSConfigSrc
  ```
- Ensure that the Kafka producer is publishing messages to every partition in a load balanced manner.
- Reduce the number of network hops between Big Data Streaming and the Kafka cluster. Ideally the Kafka broker must be on the same machine as the data node or the Kafka cluster can run on its own machines with a zero latency network.
- Configure the `batch.size` and `linger.ms` properties to increase throughput. For each partition, the producer maintains buffers of unsent records. The `batch.size` property specifies the size of the buffer. To accumulate as many messages as possible in the buffer, configure a high value for the `batch.size` property.
  By default, the buffer sends messages immediately. To increase the time that the producer waits before sending messages in a batch, set the `linger.ms` property to 5 milliseconds.
- Kafka scalability depends on disk and network performance. The test setup included 12 disks per node on a 10 GBPS network with an open file limit of 65000.

Troubleshooting Kafka Errors

While reading from Kafka, you might encounter an error similar to the following error:

```
java.lang.AssertionError: assertion failed: Failed to get records for
spark-executor-IIS-Kafka-DStream-Consumer-5c58a763-7fda-4dd1-bd84-f15dad9cd765 AggregateSrc1 6
9014479
 after polling for 512
```

This error might occur if the polling timeout limit is low. Increase the polling timeout limit.

Configure the following property for Spark in the mapping properties:

Property	Description
spark.streaming.kafka.consumer.poll.ms	Set a value between 5000 and 10000 milliseconds.

Troubleshooting Spark Driver Errors

Spark driver uses significant amount of memory when it stores the execution details of complex mappings. In such cases, you might encounter an out of memory error similar to the following error:

```
ERROR MicroBatchExecution: Query Kafka_DM_Write3 [id = d0d2cabb-767d-4525-acc7-d9edefb4beee,

runId = 3f6865fa-1619-46a0-8413-18e9c6213e5d] terminated with error

java.lang.OutOfMemoryError: Java heap space
```

This error might occur if the `spark.sql.ui.retainedExecutions` property has a higher value. To avoid such an error, set the `spark.sql.ui.retainedExecutions` property to a lower value.

Documentation Reference

See the following performance-related How-To Library articles for Informatica big data products:

- Tuning the Hardware and Hadoop Cluster for Informatica Big Data Products. Provides tuning recommendations for the hardware and the Hadoop cluster for better performance of Informatica big data products.
- Performance Tuning and Sizing Guidelines for Informatica Big Data Management 10.2.2. Provides sizing recommendations for the Hadoop cluster and the Informatica domain, tuning recommendations for various Big Data Management components, best practices to design efficient mappings, troubleshooting tips, and case studies.

Authors

Sumeet Sharma

Tanvi Sandeep